



ESCUELA COLOMBIANA DE INGENIERIA

SEMANA 5

DOSW 1



DOSW COMPANY

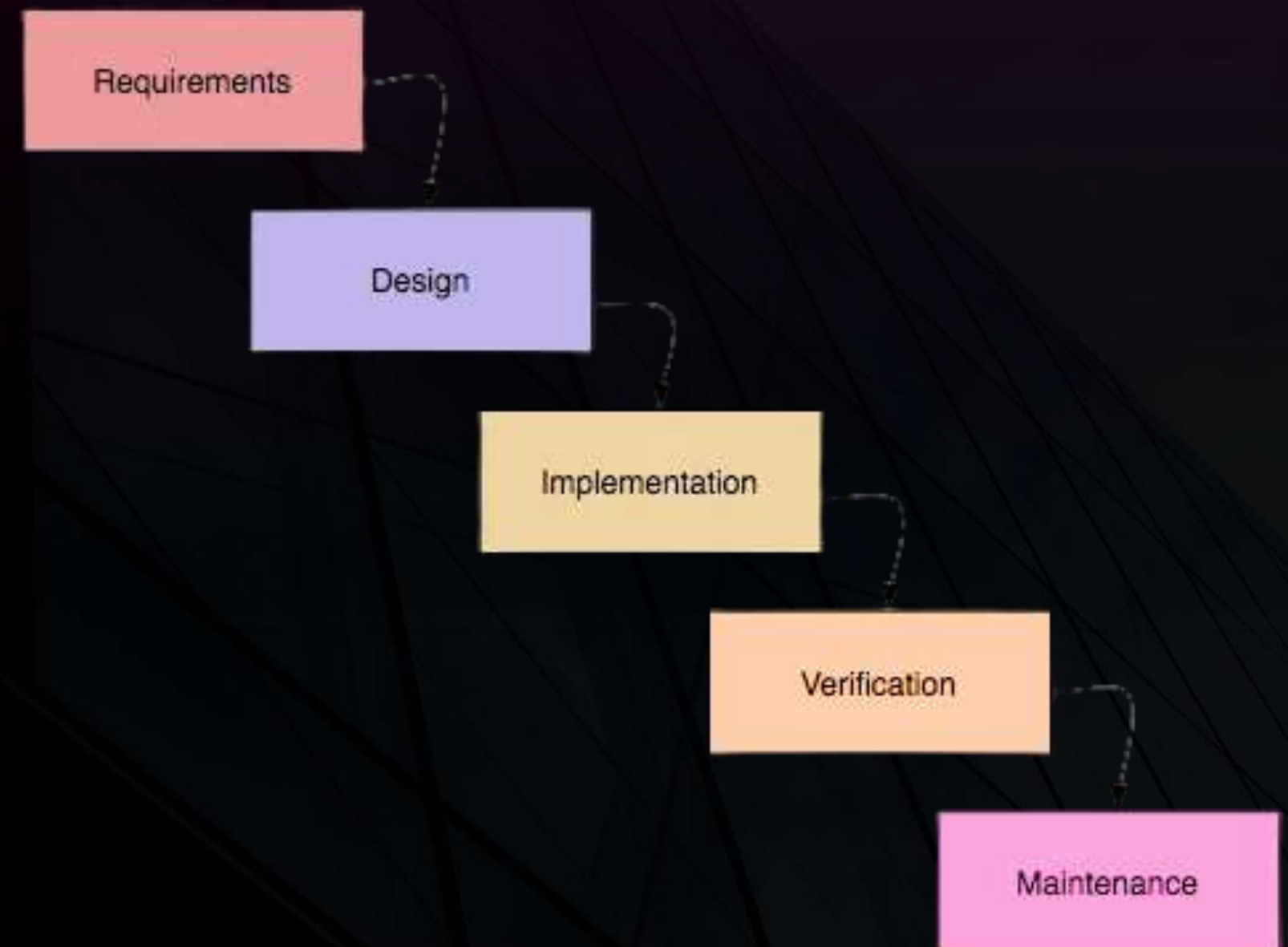
AGILISMO

¿QUÉ ES LO TRADICIONAL?

Waterfall (Cascada)

Es un enfoque de gestión de proyectos con diferentes fases que se ejecutan de manera secuencial

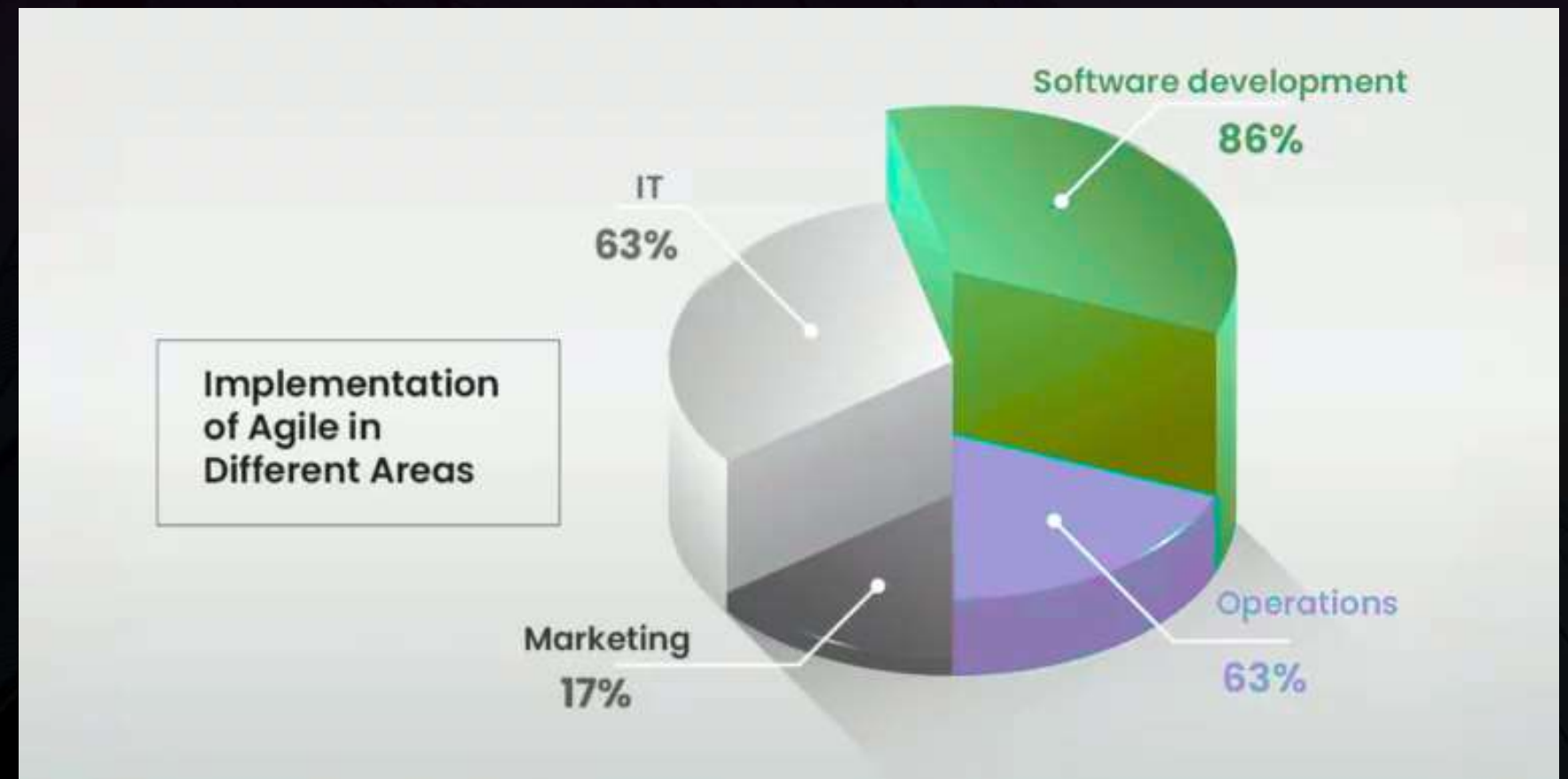
- Proyectos con requerimientos muy bien definidos y resultados predecibles.
- Proyectos con requerimientos fijos y pocos cambios.



¿QUÉ SON LAS METODOLOGÍAS ÁGILES?

“Is an approach that divides work into phases, emphasizing **continuous delivery** and **improvement**”.

–Atlassian–



–Ailotte. Diciembre 2025–

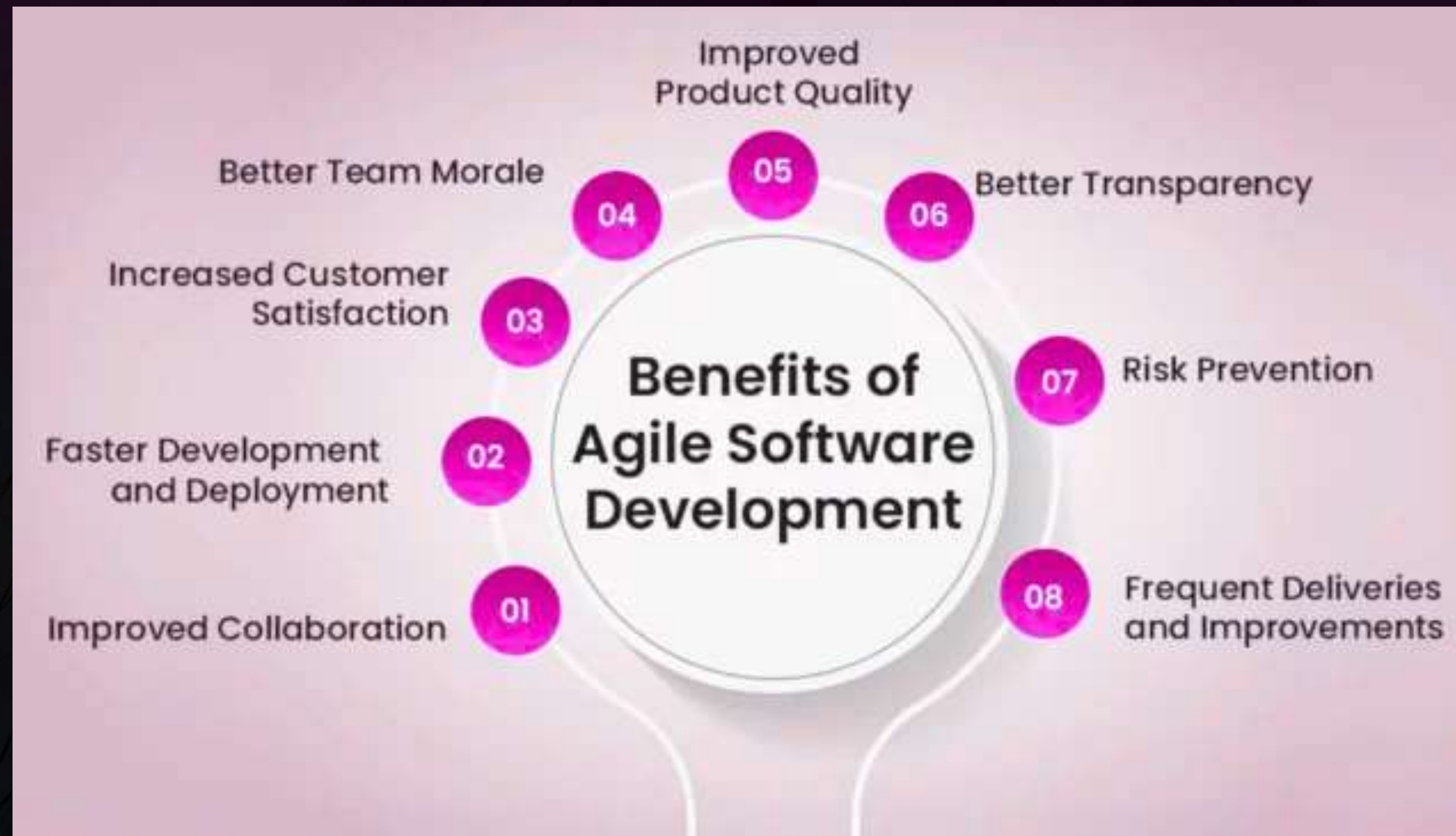
¿QUÉ SON LAS METODOLOGÍAS ÁGILES?

Capacidad de crear y responder al **cambio**, lidiando y teniendo éxito en un entorno incierto o caótico.

Divide el proyecto en pequeños segmentos (historias de usuario)

- 😊 Mejora la colaboración y productividad.
- 😊 Identifica y remueve riesgos potenciales.
- 😊 Entrega de productos de calidad.
- 😊 Simplifica la planeación.
- 😊 Ayuda a prevenir errores y re-procesos.
- 😊 Reduce costos.

BENEFICIOS



DESVENTAJAS



AGILE

Capacidad de crear y responder al cambio, lidiando y teniendo éxito en un entorno incierto o caótico

70% de empresas están en transformación ágil, acelerando la entrega de productos en un 64%

MANIFIESTO AGILE

Creado el 17 de febrero de 2001

Es una declaración de valores y principios, declarados por un grupo de expertos en desarrollo de software, que busca ofrecer una alternativa a los métodos tradicionales de gestión de proyectos.

Establecen 12 principios y 4 aspectos o pilares





DOSW COMPANY

MANIFIESTO AGILE

PRINCIPIOS



Satisfacer a los clientes



Requisitos cambiantes



Entrega de valor frecuente



Colaboración diaria



Equipos motivados



Comunicación cara a cara



Medición del progreso



Ritmo de trabajo sostenible



Excelencia técnica



Simplicidad



Equipos autoorganizados



Mejora continua



MANIFIESTO AGILE PILARES

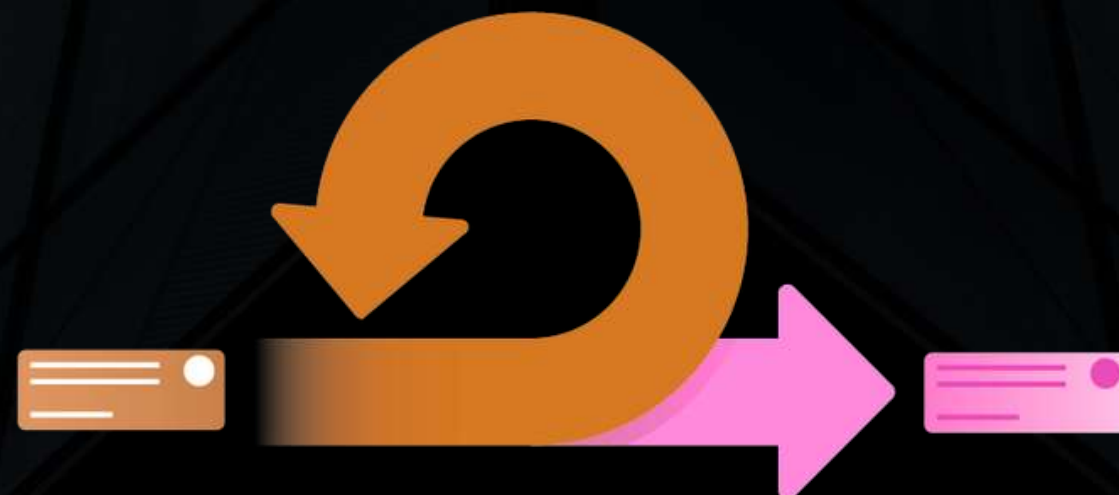


SCRUM

Marco de trabajo liviano que ayuda a las personas, equipos y organizaciones a generar valor a traves de soluciones adaptativas a problemas complejos

Usado no solamente en software ya que es efectivo en transferencia iterativa e incremental del conocimiento

Scrum se basa en el EMPIRISMO es decir se basa en tomar decisiones basado en las observaciones para aplicar los cambios en el mercado, su progreso es en base a la observacion y la experimentacion





PILARES SCRUM

TRANSPARENCIA

Mostrando y no ocultando los problemas emergentes

INSPECCION

Sobre eventos y artefactos SCRUM, así siempre se lleva a una oportunidad de mejora

ADAPTACION

Ajuste a los cambios, lo antes posible para minimizar una mayor desviación



VALORES SCRUM

CORAJE

Tratar de hacer lo correcto, mejora constante

ENFOQUE

Focalizacion, hacer algo significativo

COMPROMISO

Logro de los objetivos

RESPETO

Entre miembros del equipo

APERTURA

Transparencia, abierto al progreso y trabajo en equipo



ROLES SCRUM

SCRUM MASTER

Es el facilitador del proceso de desarrollo de Scrum. Además de celebrar reuniones diarias con el Scrum Team, el Scrum Master se asegura de que las reglas de Scrum se hagan cumplir y se apliquen según lo previsto, también es el encargado de entrenar y motivar al equipo, eliminar los impedimentos para los sprints y garantizar que el Equipo tenga las mejores condiciones posibles para cumplir sus objetivos y producir productos entregables.

PRODUCT OWNER

Representa a las partes interesadas, que suelen ser clientes. El Product Owner (Dueño del Producto) determina las expectativas del producto, registra los cambios en el producto y administra una lista de tareas pendientes detallada y constantemente actualizada para el proyecto. El Product Owner también es responsable de priorizar los objetivos de cada sprint.

SCRUM TEAM

Es un grupo autoorganizado de 3 a 9 personas que tienen las habilidades comerciales, de diseño, analíticas y de desarrollo para llevar a cabo el trabajo real, resolver problemas y producir productos entregables. Los miembros del Scrum Team se autoadministran las tareas y son conjuntamente responsables de cumplir con los objetivos de cada sprint.



ARTEFACTOS SCRUM

PRODUCT BACKLOG

Es una lista ordenada de elementos y características que podrían ser necesarios en el producto final del proyecto. Es una fuente única de requisitos. Se actualiza a medida que se modifican o agregan nuevos requisitos, correcciones, funciones y detalles en función de la nueva información y la lista de requisitos que obtengan de los clientes o usuarios.

SPRINT BACKLOG

Es la lista de tareas extraídas del Product Backlog con las que el Scrum Team se ha comprometido durante la planificación del sprint. No debes agregar trabajo a la lista de actividades pendientes del sprint a la mitad de este. Si lo estás haciendo vas a desbarrancar el proyecto, dedica más tiempo a la fase de planificación del sprint para tener una idea concreta de qué trabajo realizarás durante los sprints siguientes.

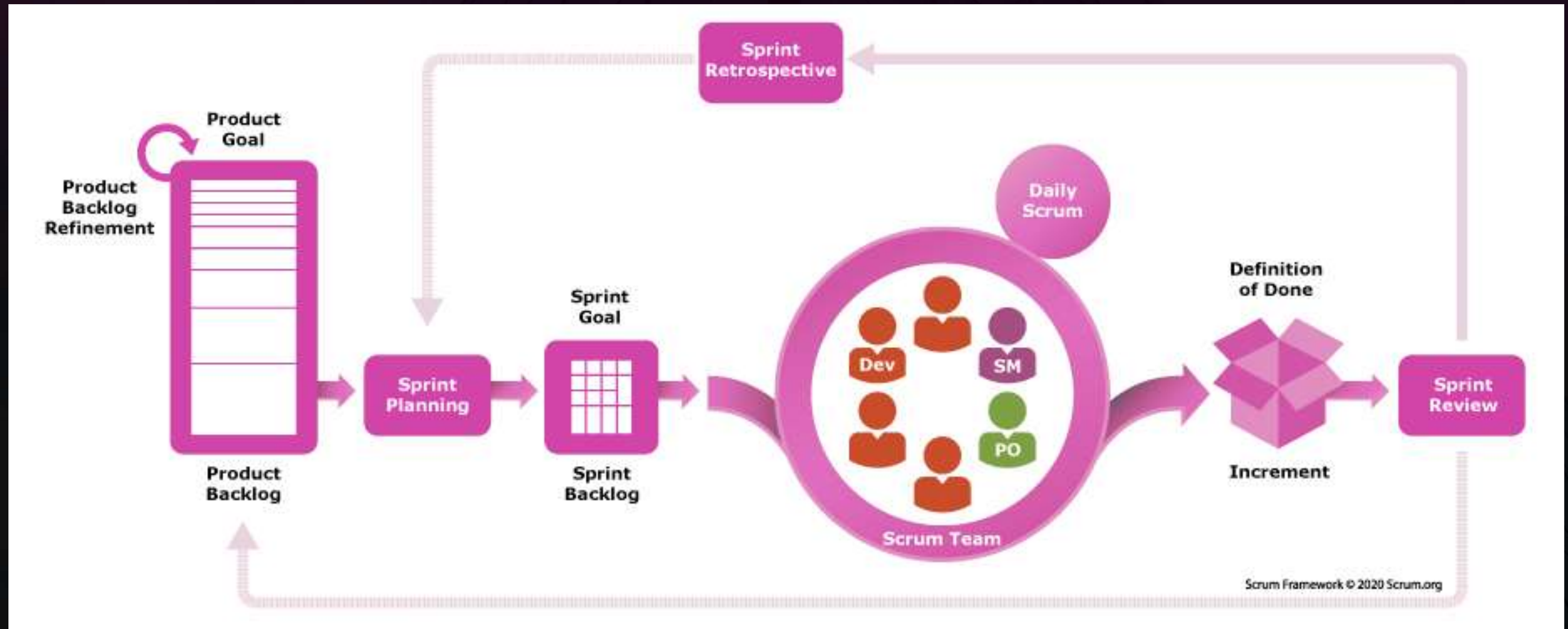
PRODUCTO INCREMENTAL

Es lo que se entrega al final de cada sprint. Este artefacto generalmente consiste en un nuevo producto o función, una mejora o corrección de errores, o cualquier otra cosa que haya solicitado el usuario o cliente y en ocasiones el mismo Scrum Team.



DOSW COMPANY

CEREMONIAS SCRUM





CEREMONIAS SCRUM

1

ORGANIZACION
DEL TRABAJO
PENDIENTE

2

PLANIFICACION
DEL SPRINT
(SPRINT
PLANNING)

3

EJECUCION DEL
SPRINT



DOSW COMPANY

CEREMONIAS SCRUM

4

REUNION
DIARIA (DAILY)

5

SPRINT REVIEW

6

SPRINT
RETROSPECTIVE



HERRAMIENTAS PARA AGILE

Top 3



Jira
(Atlassian)



Azure boards
(Microsoft)



GitLab
(GitLab)



DOSW COMPANY

ATRIBUTOS DE CALIDAD



¿QUE SON?

La calidad es un conjunto de propiedades inherentes a algo que permiten determinar su valor según el contexto, esa calidad se evalúa mediante atributos específicos que permiten medir funcionalidades y condiciones no funcionales

Ayudan a abordar la complejidad del desarrollo de software, ya que cada atributo influye en el funcionamiento e impacto final del producto

FUNCIONAL

USABILIDAD

SEGURIDAD

PERFORMANCE

FIABILIDAD

PORTABILIDAD

COMPATIBILIDAD

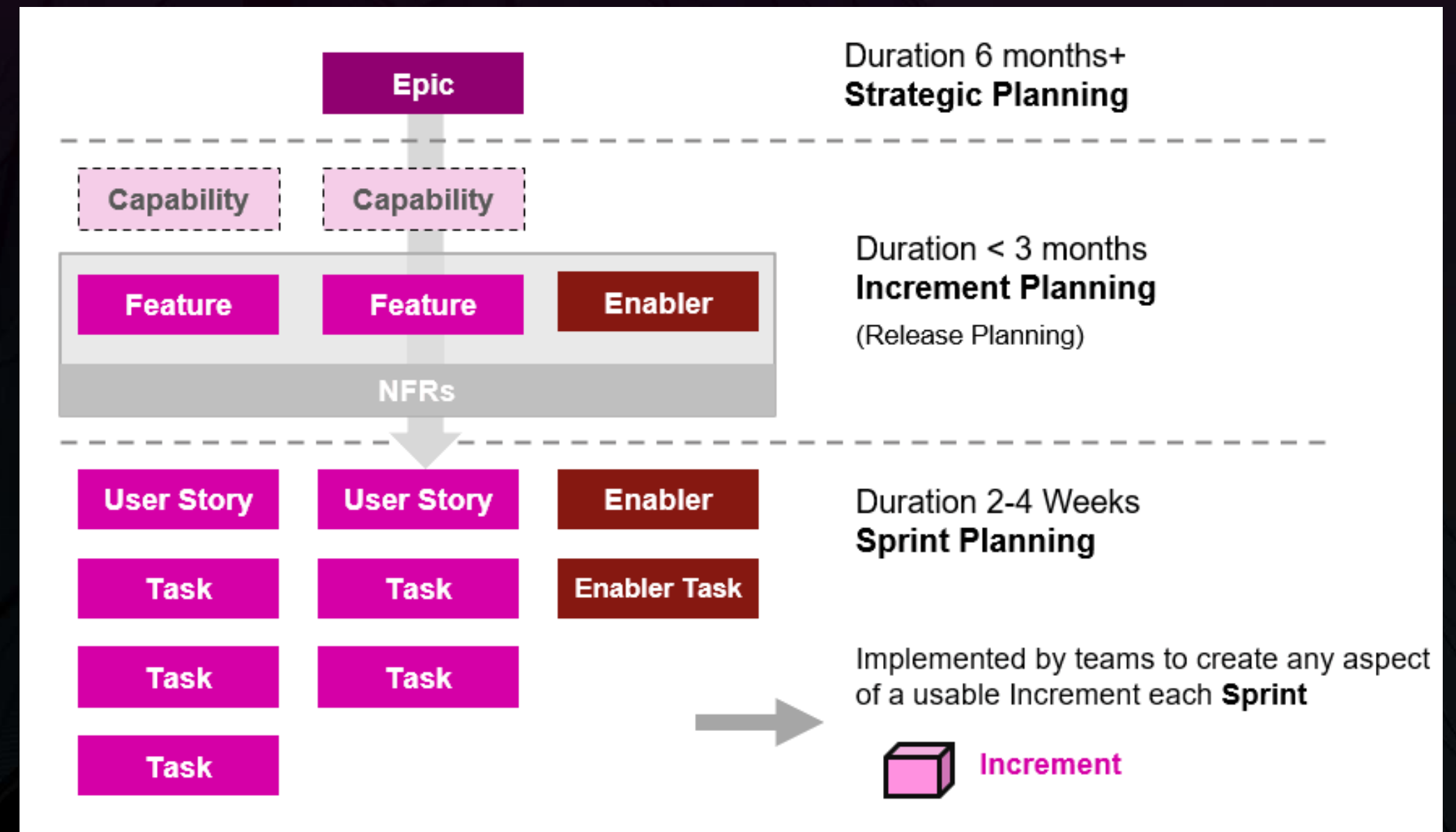
MANTENIBILIDAD



EPICA - FEATURE /HISTORIAS DE USUARIO Y TAREAS

AGILE: NIVELES DE DESCOMPOSICIÓN DEL TRABAJO

Niveles de desglose del trabajo: desde lo más grande y estratégico hasta lo más pequeño y operativo.





EPICA

Es una gran iniciativa o necesidad de negocio que describe un objetivo amplio. Es demasiado grande para ser desarrollada en un solo sprint, por lo que debe dividirse en features y user stories

¿COMO SE GENERA?

- Se origina desde la vision del producto o un objetivo estrategico de negocio
 - Normalmente la redactan product owners o stakeholders
 - Sirve como contenedor de funcionalidades relacionadas

EJEMPLO

“Un cliente de un banco digital desea poder realizar pagos moviles para no depender de ir a una sucursal fisica”

“Sistema para gestionar horarios”



FEATURE

Es una funcionalidad concreta que aporta valor y ayuda a cumplir la epica. Puede completarse en varios sprints, pero es mas pequeña y especifica que la epica

¿COMO SE GENERA?

- Se deriva de la epica dividiendo en partes mas manejables
- La define normalmente el product owner junto con el equipo tecnico
 - Cada feature se descompone en historias de usuario

EJEMPLO

“Implementar pagos a traves de codigo QR desde la aplicacion movil”

“Modulo de gestion de materias”



HISTORIA DE USUARIO

Una historia de usuario es la unidad más pequeña de trabajo en un backlog ágil. Representa una necesidad específica del usuario final, escrita desde su perspectiva.

¿COMO SE GENERA?

- Se deriva de una feature.
- Se redacta en formato: Como [rol], quiero [funcionalidad], para [beneficio]
- La complementan criterios de aceptación para saber cuándo está terminada.

EJEMPLO

“Como cliente quiero escanear un código QR desde la cámara de mi celular para pagar rápidamente sin ingresar datos manuales”

Como Director de Centro quiero crear una materia en el sistema.



TAREA

Son las acciones técnicas específicas necesarias para completar una historia de usuario.

¿COMO SE GENERA?

Nivel alto de detalle
Asignable a una persona

EJEMPLO

- Crear la tabla materia con los datos ...
- Implementar la lógica para crear una materia.
 - Integrar la base de datos con el backed.
- Implementar la pantalla para la creación de una materia.

AGILE: NIVELES DE DESCOMPOSICIÓN DEL TRABAJO

Resumen visual

Nivel	Enfoque	¿Quién lo define?	Tamaño
Épica	Estrategia	Negocio / Producto	Muy grande
Feature	Funcionalidad	Producto	Grande
Historia de Usuario	Necesidad del usuario	Producto + Equipo	Mediano
Tareas	Implementación	Equipo técnico	Pequeño



ACTIVIDAD

Proyecto:

DOSW Fest – Organicemos un festival universitario

Misión:

- DOSW Company fue contratada para organizar el DOSW FEST 2026, un festival universitario que se realizará dentro del campus.
- Asistirán aproximadamente 500 personas y habrá:
- 🎸 bandas y DJs
- 🍔 puestos de comida
- 🎮 torneo de videojuegos
- 🎤 karaoke
- 🏆 concursos
- 🎫 ingreso mediante entradas
- 🎁 premios

Fecha: 31 de octubre de 2026

Número de invitados: 500



ACTIVIDAD

Proyecto:

DOSW Fest – Organicemos un festival universitario

- Solo pueden ingresar personas previamente registradas.
- Cada asistente recibe un QR de entrada y solo puede utilizarse una vez.
- Debe existir control de aforo.
- Los estudiantes pueden inscribirse en actividades y concursos.
- Una persona no puede participar en dos actividades que ocurran simultáneamente.
- Las bandas deben conocer escenario y horario.
- Los puestos de comida deben tener un espacio asignado.
- Los asistentes deben poder conocer la programación.
- Si una actividad cambia de horario, sus participantes deben enterarse.
- Los ganadores de concursos reciben premios.
- La organización necesita conocer en tiempo real cuántas personas ingresaron.
- A las 10:00 p. m. debe terminar el evento.



PRIMERA ETAPA - CUAL ES LA EPICA?

Nombre: DOSW FEST 2026

Gestionar integralmente la experiencia y operación del DOSW FEST 2026, desde el registro de los asistentes hasta la ejecución y cierre del evento.



Identifiquen entre 4 y 7 grandes funcionalidades necesarias para cumplir la
Épica.

SEGUNDA ETAPA - FUNCIONALIDADES

1. Control de acceso al Festival

¿QUIÉN OBTIENE VALOR DE ESTA
FEATURE?

Asistentes



Personal de Acceso



Administrador



Ahora conviertan la Feature en necesidades de esos
usuarios.

SEGUNDA ETAPA - FUNCIONALIDADES

1. Control de acceso al Festival



Como asistente, quiero recibir una entrada digital con QR para ingresar al festival



Como personal de acceso, quiero escanear el QR de un asistente para validar que puede ingresar.

SEGUNDA ETAPA - FUNCIONALIDADES

1. Control de acceso al Festival



Como organizador, quiero consultar el número de personas que se encuentran dentro del evento para controlar el aforo.



Como personal de acceso, quiero conocer cuando un QR ya fue utilizado para evitar ingresos duplicados.



TERCERA ETAPA - CRITERIOS DE ACEPTACION

1. Como personal de acceso quiero escanear el QR de un asistente para validar su entrada al DOSW FEST.



¿Cuándo consideramos terminada esta HU?

DADO que un asistente tiene una entrada válida
CUANDO el personal escanea su QR
ENTONCES el sistema debe autorizar su ingreso.

Yo vuelvo a entrar con el mismo QR entonces...

DADO que un QR ya fue utilizado
CUANDO se intenta escanear nuevamente
ENTONCES el sistema debe rechazar el ingreso
Y mostrar que la entrada ya fue utilizada.

TERCERA ETAPA - CRITERIOS DE ACEPTACION



Los criterios de aceptación no son
decoración: representan reglas
verificables del negocio.

CUARTA ETAPA - TAREAS

Tenemos esta HU. ¿Qué tenemos que construir para hacerla realidad?



- HU — Validar ingreso mediante QR
 - |
 - └─ TASK — Diseñar modelo Ticket
 - └─ TASK — Implementar generación de QR
 - └─ TASK — Implementar endpoint de validación
 - └─ TASK — Implementar validación de estado del ticket
 - └─ TASK — Implementar control de doble ingreso
 - └─ TASK — Crear interfaz de escaneo
 - └─ TASK — Integrar scanner con backend
 - └─ TASK — Crear manejo de errores
 - └─ TASK — Crear pruebas unitarias
 - └─ TASK — Crear pruebas de integración



CUARTA ETAPA - TAREAS



Hasta llegar a TASKS están prohibidas las palabras: endpoint, controller, database, React, Java, Spring, API.

ÉPICA / FEATURE / HU → HABLAN DEL PROBLEMA Y DEL VALOR.
TASKS → HABLAN DE CONSTRUCCIÓN.

REFINAMIENTO

EVENTO #1: Un estudiante le tomó screenshot al QR de su amigo y está intentando entrar con él.

EVENTO #2: Llegamos al aforo máximo de 500 personas.

EVENTO #3: La banda principal canceló 30 minutos antes de su presentación.

EVENTO #4: El torneo de videojuegos cambió de 4:00 p. m. a 6:00 p. m., pero 40 personas estaban inscritas.

EVENTO #5: Una persona intenta registrarse en karaoke a las 5:00 p. m. y FIFA a las 5:00 p. m.



¿NECESITAMOS UNA FEATURE NUEVA?
¿UNA HU NUEVA?
¿UN NUEVO CRITERIO DE
ACEPTACIÓN?
¿O ESTO YA ESTABA CUBIERTO?

BACKLOG COURT

Hagan equipos de 4 personas

Cada equipo presenta 1 Feature y sus HU

El resto del curso es el Backlog Court

Despues de explicada cada equipo de la corte puede
levantar una tarjeta:

-  **RECHAZADA** — No es una HU.
-  **SPLIT** — Es demasiado grande, divídala.
-  **FEATURE** — Eso que llamaron HU realmente parece Feature.
-  **TASK** — Están describiendo implementación.
-  **READY** — Podría entrar al Sprint.

Pero quien levante la tarjeta tiene que justificarla.





DOSW COMPANY

JIRA



SU MISIÓN... TALLER #6

Taller enfocado en como podemos desglosar y organizar en EPICA/Features/HU y Tareas para el proyecto de TECHCUP

Crear el JIRA para este ejercicio

**DEBE IR EN LA BITACORA
EL LINK DE JIRA**



DOSW COMPANY

ESTIMACION DE TAREAS



EN QUE CONSISTE

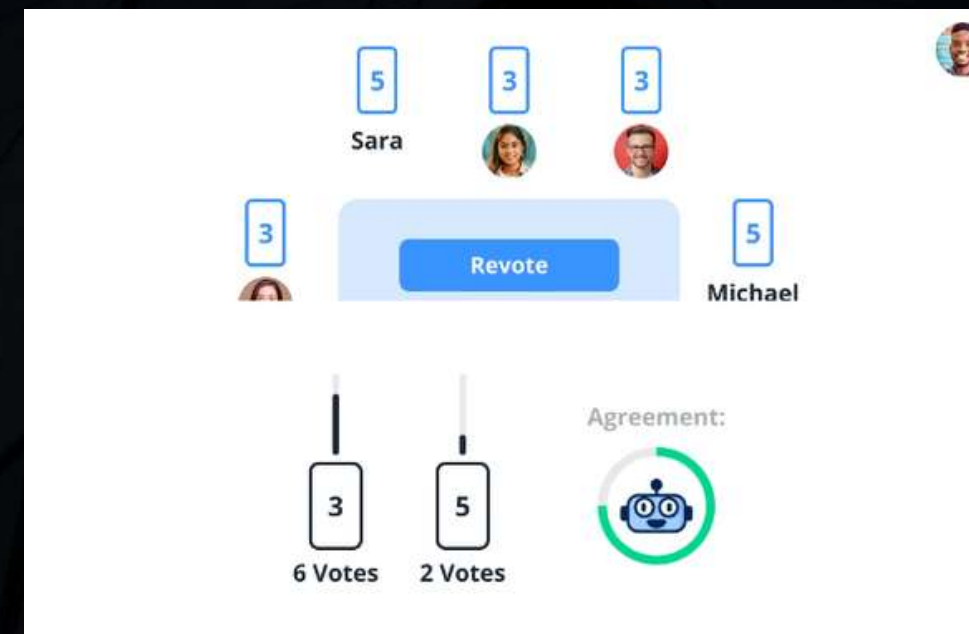
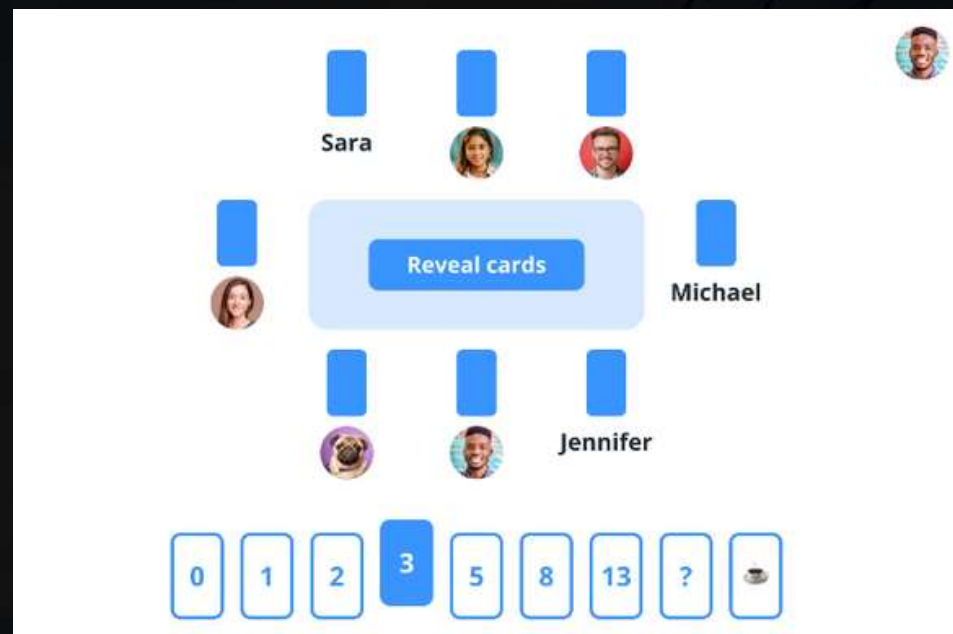
En Scrum, la estimación se refiere al proceso de asignar valores relativos a las tareas o historias de usuario para determinar su complejidad o esfuerzo requerido. Estas estimaciones se utilizan para planificar los sprints y predecir la velocidad del equipo. Las estimaciones se realizan típicamente en "puntos de historia", utilizando técnicas como PLANNING POKER para llegar a un consenso

ES CRUCIAL QUE TODOS LOS MIEMBROS DEL EQUIPO DE DESARROLLO PARTICIPEN EN LA ESTIMACION



PLANNING POKER

- Los participantes emplean cartas con una numeración especial que asignan a cada uno de los elementos del Product Backlog. Después de estimar un elemento del Product Backlog, las personas que han otorgado la mayor y la menor puntuación explican los motivos por los que han escogido dicha puntuación y se vuelve a realizar la estimación, así hasta que se llegue a un consenso.
- La numeración de las cartas está basada en una serie de Fibonacci (0, 1, 2, 3, 5, 8, 13, 21 34, etc.), aunque en ocasiones se pueden variar algunos números de la serie.





DOSW COMPANY

COMUNICACION PARA EQUIPOS





DOSW COMPANY

PRUEBAS



PRUEBAS DE SOFTWARE

Son el proceso de evaluar y verificar que un producto o aplicación de software hace lo que se supone que debe hacer.

Unas buenas pruebas de software permiten la prevención de errores (pero no al 100%) y la mejora del rendimiento

Son más efectivas cuando se realizan de forma continua, lo que indica que se inician durante el diseño, continúan durante la construcción e incluso cuando el software ya está en producción

SON IMPORTANTES PORQUE

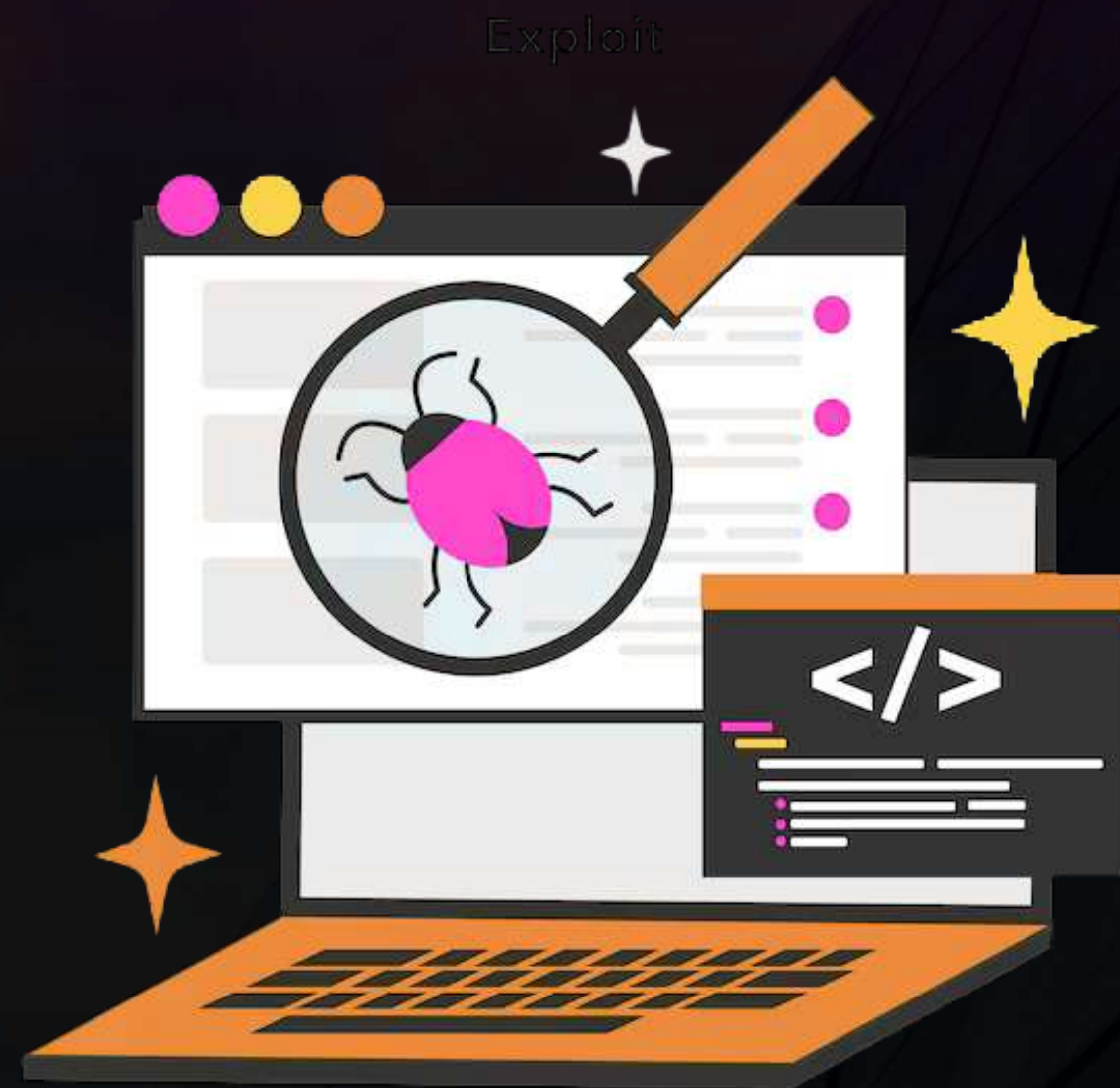


Permiten identificar defectos en la arquitectura, identificar decisiones de diseño poco acertadas, funcionalidades no validas o incorrectas, vulnerabilidades y problemas en las aplicaciones



DOSW COMPANY

¿QUE ES UNA VULNERABILIDAD?



Es una debilidad existente en un sistema que puede ser utilizada por un actor malintencionado para comprometer su seguridad

Pueden ser de varios tipos, desde hardware hasta software, una vulnerabilidad puede ser:

- ❌ Un servicio corriendo en un determinado puerto logico
- ❌ Sistemas y aplicaciones no actualizados o parcheados que presentan multiples problemas
- ❌ Una red wifi abierta
- ❌ La no aplicacion de una politica de gestion de contraseñas



DOSW COMPANY

TIPOS DE PRUEBAS



PRUEBAS UNITARIAS

Son pruebas de bajo nivel, realizadas cerca de la fuente de la aplicación, prueba métodos y funciones individuales de las clases, componentes o módulos del software

EJEMPLO: Probar el método suma de una calculadora



PRUEBAS FUNCIONALES

Se centran en los requisitos empresariales de una aplicación, solo verifican el resultado de una acción y no comprueban estados intermedios del sistema al realizar dicha acción

EJEMPLO: Probar el Login de una aplicación



PRUEBAS DE CARGA

Simulan una demanda de tráfico esperada en una aplicación para evaluar su rendimiento, capacidad de respuesta, tasas de rendimiento y utilización de recursos bajo condiciones de estrés

EJEMPLO: Probar el Flujo de transacciones para una pasarela de pagos en Black Friday

PRUEBAS DE CAJA NEGRA

Verifican el comportamiento
de un sistema sin conocer su
codigo fuente ni estructura
interna

Pruebas Funcionales

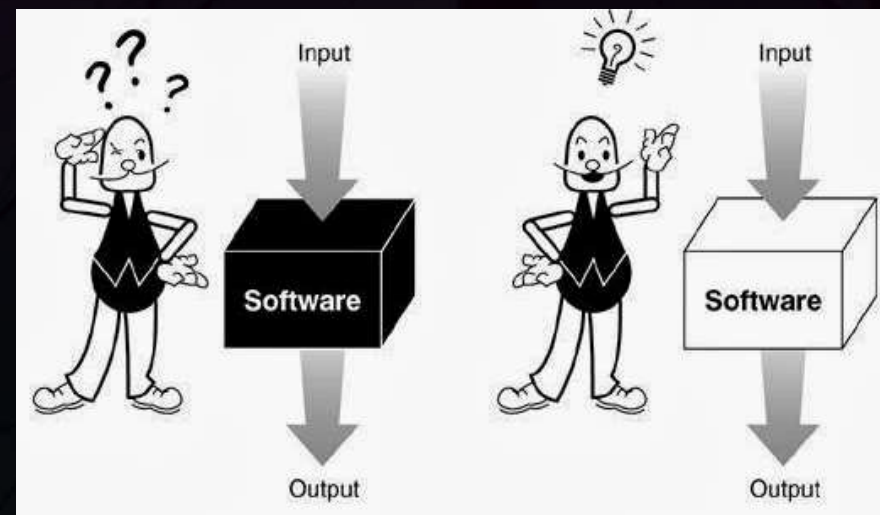
Pruebas de Carga



PRUEBAS DE CAJA BLANCA

Verifica el funcionamiento
interno de un sistema,
teniendo acceso total al
codigo fuente, la
infraestructura y los
documentos de diseño del
software en cuestion.

Pruebas unitarias





T.O.D - TEST DRIVEN DEVELOPMENT



T.D.D - TEST DRIVEN DEVELOPMENT

Es una practica de programacion que consiste en escribir primero las pruebas (generalmente unitarias), despues escribir el codigo fuente que pase la prueba satisfactoriamente y por ultimo, refactorizar el codigo escrito



- Este ciclo tambien se conoce como ROJO, VERDE y Refactor

T.D.D - EJEMPLO



- Un cliente nos pide que desarrollemos una calculadora que sume numeros
- Se acuerda con el cliente los criterios necesarios es decir: Si al introducir en la calculadora dos numeros y le das a la operacion de sumar, la calculadora debe mostrar el resultado en pantalla
- A partir de lo anterior se comienza a definir el funcionamiento del algoritmo de suma y lo convertimos en una prueba concreta, por ejemplo que pasa si sumamos $3 + 5 = 8$



```
public void testSuma() { assertEquals(8, Calculadora.suma(3,5)); }
```

T.D.D - EJEMPLO



- La clase aun no existe pero ya sabemos como va a funcionar, hay casos donde no se sabe exactamente que clases existen o que metodos se colocan, solo es necesario lo que se necesite en el momento
- Al ejecutar el test nos dara error, porque la clase aun no existe
- Ahora pasamos a escribir el codigo de la clase, es facil porque ya sabemos su comportamiento

```
public class Calculadora { public static int suma (int a, int b) { int c = a + b; return c; } }
```

- Ahora re ejecutamos la prueba, y ya debe estar funcionando gracias a la implementacion
- Una vez funcionando, pasamos a refactorizar y a eliminar codigo duplicado

```
public class Calculadora { public static int suma (int a, int b) { return a+b; } }
```

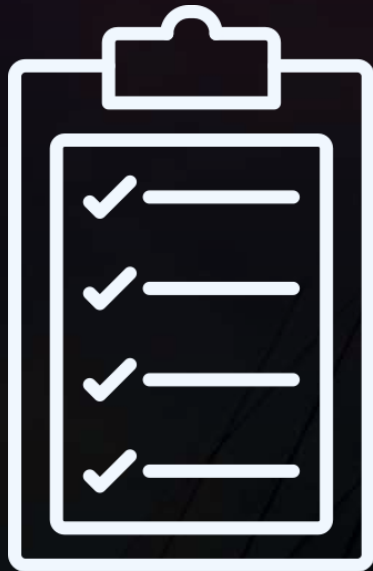


T.D.D - EJEMPLO

- Es importante pasar todos los test despues de refactorizar por si se ha cambiado algo
 - Se puede volver a los criterios de la calculadora con funcionalidades mas complicadas como lo es admitir numeros decimales, sumar numeros negativos, que la suma sin importar si $a > b$ o $b > a$ de igual, etc, y replicar el mismo procedimiento.



¿COMO DISEÑAR ESCENARIOS DE PRUEBAS?



CRITERIOS DE
ACEPTACION



PRINCIPIOS DE
LAS PRUEBAS



PARTES DE UNA
PRUEBA

CRITERIOS DE ACEPTACION



Son un conjunto de condiciones o reglas que una funcionalidad debe cumplir para considerarse terminada y aceptada por el usuario, el cliente o el Product owner (Dueño del Producto)

CARACTERÍSTICAS

Se escriben desde la perspectiva del usuario final

Definen que significa listo (Definition of Done) para un requisito

Son medibles y verificables, de forma que se pueda comprobar con pruebas

Sirven como base para las pruebas





CRITERIOS DE ACEPTACION



EJEMPLOS

- 1.El sistema debe permitir ingresar un correo y una contraseña validas
2. Si las credenciales son correctas, el usuario debe ser redirigido a la pagina principal.
- 3.Si las credenciales son incorrectas, debe mostrar un mensaje: "Usuario o contraseña incorrectos"
- 4.El campo de contraseña debe ocultar los caracteres escritos
- 5.Debe existir un enlace para recuperar la contraseña

PRINCIPIOS DE LAS PRUEBAS

7

1. La ejecución de pruebas demuestra la presencia de defectos, no confirmar que no existen
2. El testing exhaustivo no es posible, no se pueden probar todas las combinaciones y acciones en todas las funcionalidades, debido a limitaciones
3. Las Pruebas Tempranas ahorran tiempo, recursos y dinero, detectando problemas antes del lanzamiento
4. Agrupación de defectos - 80% de las consecuencias provienen del 20% de las causas - no todos los componentes tienen la misma importancia
5. Paradoja del pesticida - repetir las mismas pruebas una y otra vez no disminuirá su efectividad en la detección de errores
6. El testing depende del contexto - Se elige según el sistema y los entornos que decidan verificar (sistema médico vs un e-commerce)
7. Falacia de ausencia de incidentes

**REVISEN CONSTANTEMENTE SU
ESTRATEGIA DE PRUEBAS**



PARTES DE UNA PRUEBA

ARRANGE

Se prepara el escenario de prueba - Aqui se crean los objetos, se inicializan datos de entrada, se configuran mocks, etc

ACT

Se ejecuta la accion que se quiere probar - Normalmente es la llamada a un metodo o a una funcionalidad concreta

ASSERT

Se validan los resultados obtenidos contra los resultados esperados - aca se realizan las verificaciones assert, assertTrue, assertEquals

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraTest {

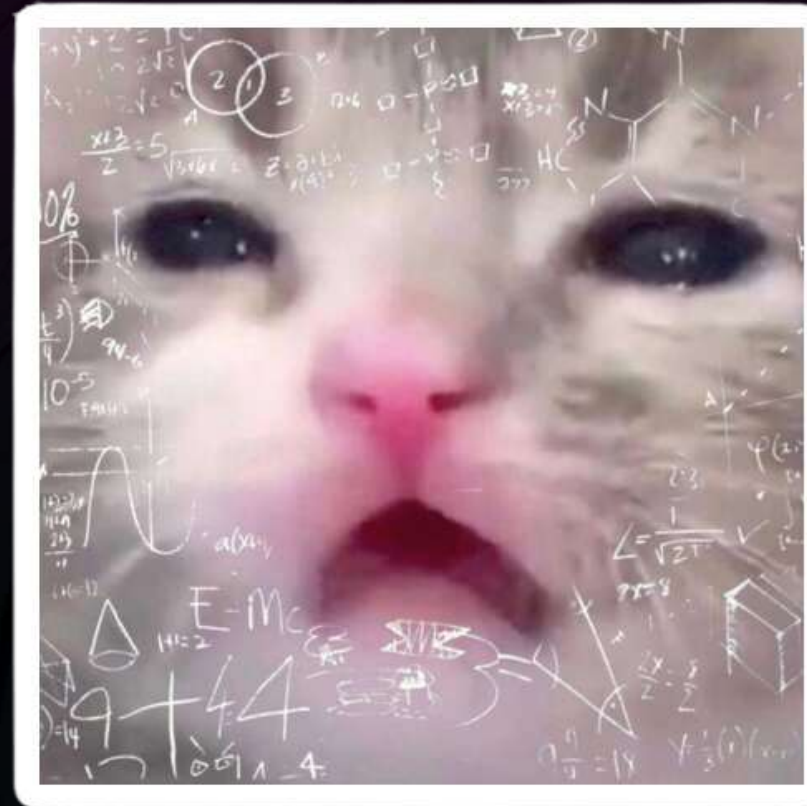
    @Test
    void testSumar() {
        // PREPARE (Arrange)
        Calculadora calc = new Calculadora();

        // EXECUTE (Act)
        int resultado = calc.sumar(2, 3);

        // ASSERT
        assertEquals(5, resultado, "La suma de 2 + 3 debe ser 5");
    }
}
```



RECUERDEN



Una buena prueba se ejecuta rapidamente
Se programan rapido
No dependen de elementos externos
No dependen de otras pruebas unitarias



COBERTURA DE PRUEBAS



¿QUE ES LA COBERTURA DE PRUEBAS?

Es una de las medidas de la calidad de los tests que nos dice cuanto codigo se ha probado

Define ciertas entidades del sistema con la intencion de cubrirlas con pruebas, siendo una forma de indicar cuanto hemos probado lo suficiente y nos da ideas de que mas probar asi ampliando la cobertura

TENER UN CIERTO NIVEL DE COBERTURA ES UN INDICADOR DE LA CALIDAD DE LAS PRUEBAS, PERO NUNCA ES UN INDICADOR DE LA CALIDAD DEL SISTEMA, NI TAMPOCO GARANTIZA QUE TODO HALLA SIDO PROBADO



JUNIT

Es el framework de pruebas unitarias mas usado en JAVA, permite automatizar pruebas de metodos y clases de manera mas rapida

¿COMO FUNCIONA?

- Usa anotaciones como @Test, @BeforeEach, @AfterEach, etc
 - Ejecuta las Pruebas de forma aislada
- Verifica resultados con asserts (assertEquals, assertTrue, etc)
- Se integra con MAVEN, Gradle, IDEs y herramientas de CI/CD

**EN MAVEN SE EJECUTA CON MVN
TEST**



DOSW COMPANY



JUNIT

DEPENDENCIA

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.9.3</version>
  <scope>test</scope>
</dependency>
```

EJEMPLO

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraTest {
    @Test
    void testSuma() {
        assertEquals(5, 2 + 3);
    }
}
```



JACOCO
Java Code Coverage

JACOCO

Es una herramienta que mide la cobertura del código de las pruebas, es el encargado de informar qué porcentaje de código fue ejecutado durante las pruebas

¿COMO FUNCIONA?

- Se ejecuta junto con JUnit u otro framework de pruebas
- Instrumenta el bytecode para registrar qué líneas, ramas y métodos se ejecutan
 - Genera reportes en HTML, XML o CSV

EN MAVEN SE EJECUTA CON `MVN TEST` Y SE VE EL REPORTE EN `TARGET/SITE/JACOCO/INDEX.HTML`



JACOCO
Java Code Coverage

JACOCO

PLUGIN

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.10</version>
  <executions>
    <execution>
      <goals>
        <goal>prepare-agent</goal>
      </goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals>
        <goal>report</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```



The SonarQube logo, consisting of the word "sonarqube" in a black, lowercase sans-serif font, followed by three blue curved lines representing a sonar wave.

SONAR QUBE

Es una herramienta de analisis estatico de codigo que ayuda a mantener la calidad del software, analiza: Bugs, Code Smells (malas practicas), Vulnerabilidades de seguridad, Cobertura de pruebas, Duplicacion deCodigo

¿COMO FUNCIONA?

- Escanea el proyecto (JAVA, JS, Python, etc)
- Genera metricas y las guarda en un servidor
- Muestra un dashboard web con los resultados
- Se integra con CI/CD (Jenkins, Gitlab, github actions, etc)

SE INSTALA SONAR QUBE SERVER LOCAL O EN DOCKER

```
docker run -d --name sonarqube -p 9000:9000 sonarqube
```



sonarqube

SONAR QUBE

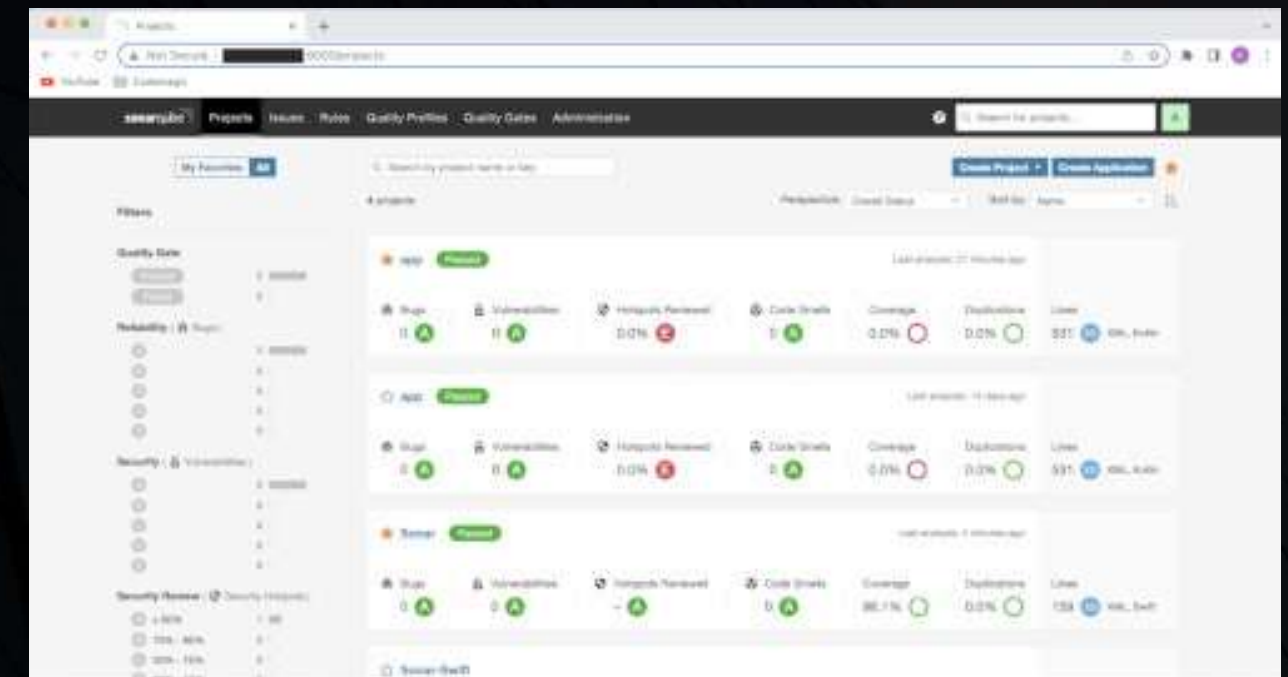
PLUGIN

```
<plugin>  
  <groupId>org.sonarsource.scanner.maven</groupId>  
  <artifactId>sonar-maven-plugin</artifactId>  
  <version>3.10.0.2594</version>  
</plugin>
```

EJECUCION

```
mvn clean verify sonar:sonar \  
-Dsonar.projectKey=mi-proyecto \  
-Dsonar.host.url=http://localhost:9000 \  
-Dsonar.login=TOKEN
```

DASHBOARD





DOSW COMPANY

MOCKS Y STUBS



DOSW COMPANY

MOCKS

¿QUE SON?

Es un objeto que registra las llamadas a sus metodos y valida si ocurrieron las interacciones esperadas, se usan cuando se quiere comprobar como colabora tu objeto con otros componentes, son mas que observadores ya que pueden lanzar excepciones

Pueden Verificar que los metodos fueron invocados, en que orden y cuantas veces

Ideales para pruebas de comportamientos

MOCKS

VENTAJAS

- Validan interacciones entre los componentes
- Detectan efectos colaterales ocultos (no visibles en valores de retorno)
- Aseguran que la colaboracion entre objetos es la esperada



DESVENTAJAS

- Su uso excesivo genera test fragiles, acoplados a la implementacion
- Pueden agregar complejidad innecesaria a la suit de pruebas





DOSW COMPANY

STUBS

¿QUE SON?

Es un componente que devuelve respuestas predefinidas durante un prueba, se usan cuando importa mas el comportamiento del objeto bajo prueba que sus interacciones con otros componentes

Es como un actor con guion que simula escenarios (Transaccion Aprobada/Rechazada)

Las respuestas estan preprogramadas (valores fijos), por lo que son predecibles y repetibles

VENTAJAS

Aislan la unidad bajo pruebas de dependencias externas, Resultados predecibles y permiten probar la robustez del codigo en diferentes escenarios

DESVENTAJAS

Si no se configuran bien dan una falsa sensacion de correcto, no capturan informacion de interaccion solo resultados



MOCKS Y STUBS USO CON MOCKITO

CODIGO BASE

```
public class UserService {  
    private final UserRepository userRepository;  
  
    public UserService(UserRepository userRepository) {  
        this.userRepository = userRepository;  
    }  
  
    public String getUserEmail(int userId) {  
        User user = userRepository.findById(userId);  
        return user.getEmail();  
    }  
}
```

PRUEBA CON STUB

```
@Test  
void testGetUserEmail_withStub() {  
    // Creamos un stub manual  
    UserRepository userRepositoryStub = new UserRepository() {  
        @Override  
        public User findById(int userId) {  
            return new User(userId, "test@example.com");  
        }  
    };  
  
    UserService userService = new UserService(userRepositoryStub);  
  
    String email = userService.getUserEmail(1);  
  
    assertEquals("test@example.com", email);  
}
```

PRUEBA CON MOCK

```
import static org.mockito.Mockito.*;  
  
@Test  
void testGetUserEmail_withMock() {  
    // Creamos un mock del repositorio  
    UserRepository userRepositoryMock = mock(UserRepository.class);  
  
    // Definimos qué debe devolver cuando se llame con 1  
    when(userRepositoryMock.findById(1))  
        .thenReturn(new User(1, "mock@example.com"));  
  
    UserService userService = new UserService(userRepositoryMock);  
  
    String email = userService.getUserEmail(1);  
  
    assertEquals("mock@example.com", email);  
  
    // Verificamos la interacción  
    verify(userRepositoryMock, times(1)).findById(1);  
}
```



BUENAS PRACTICAS

- ✓ Equilibrar Stubs y Mocks, no usen solamente uno siempre
- ✓ Usar Stubs cuando lo importante son los resultados (state testing)
- ✓ Usar Mocks cuando lo importante son las interacciones (behavior testing)
- ✓ Evitar tests demasiado especificos (se vuelven fragiles ante pequeños cambios)
- ✓ Mantener los test claros, simples y legibles
- ✓ En TDD permiten probar antes de tener implementaciones reales



LABORATORIO #3

¿Como les ha ido con el laboratorio?

¿Tienen dudas?

¿Que tan complejo sientieron los retos propuestos?

CUIDADO CON el orden del README.md





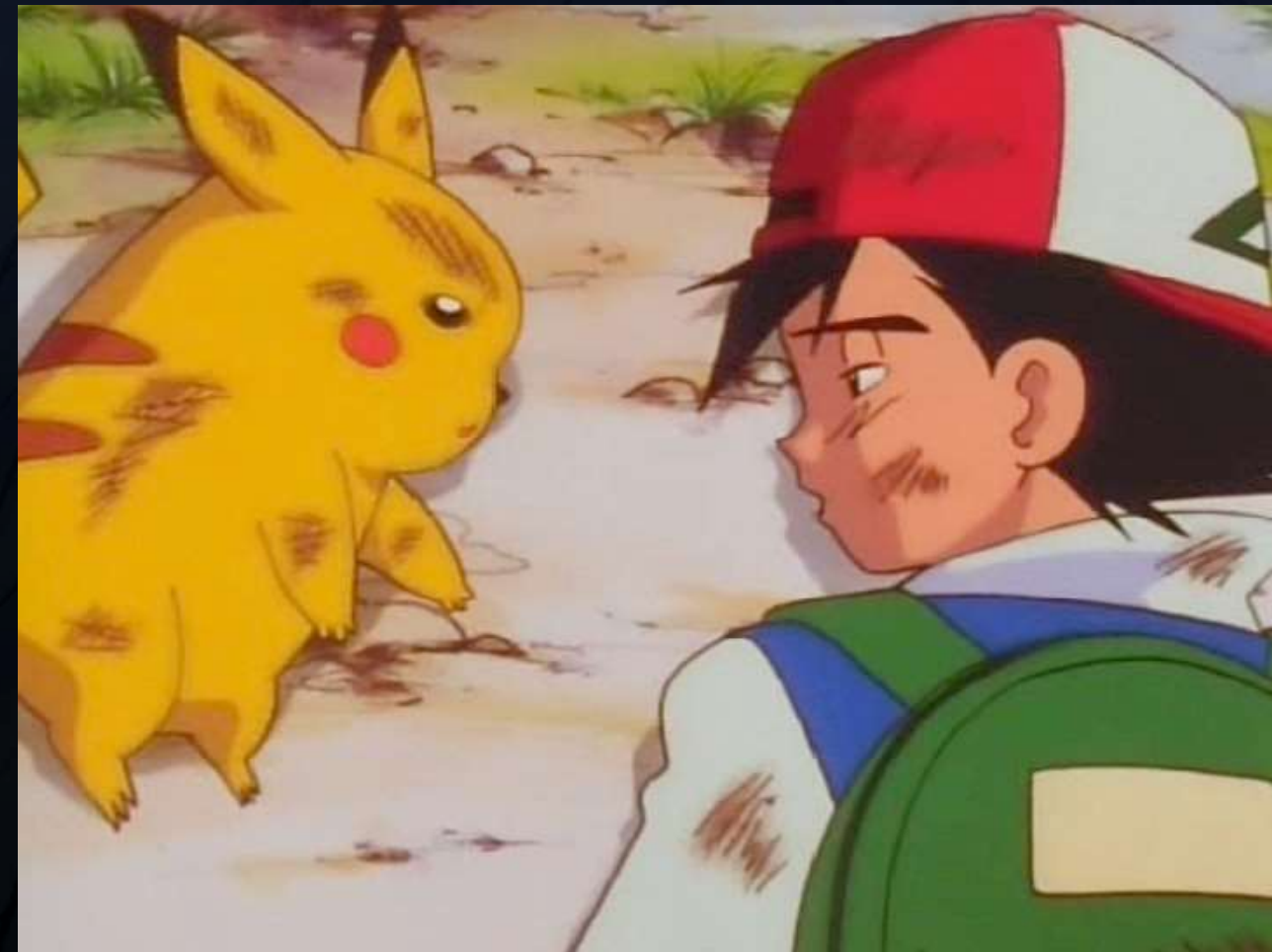
TAREA

1.TALLER #6 - ENTREGA 27 DE AGOSTO

1.VEAN VIDEOS RELACIONADOS CON LA ESTIMACION DE
TAREAS UTILIZANDO PLANING POKER



DOSW COMPANY



NOS VEMOS EN LA
PROXIMA SESION DE
LABORATORIO